ASSIGNMENT 3

Dynamic Student Portfolio Web Application Using Amazon EC2, Amazon RDS MySQL and Amazon DynamoDB

Detail	Information
Student Name	Malaganti Pardha Surya Chakravarthi
Registration No.	24BCA8072
Program	B.Tech Artificial Intelligence and Machine Learning
University	VIT-AP University, Amaravati
Venue	CB-416
Slot	B2 + TB2
Live Website	http://52.55.224.145/
Date	09 October 2026

School of Computer Science and Engineering
VIT-AP University, Amaravati

1. Abstract

This report describes the design, deployment and testing of a dynamic student portfolio web application hosted on Amazon EC2. The application shows how one web tier can use two different AWS database services, each chosen for the type of data it stores. The front end uses a retro game-inspired HTML, CSS and JavaScript design and is served by the Apache web server on an Ubuntu Linux EC2 instance. The dynamic pages are written in PHP.

Structured academic records (course code, subject, credits, grade and semester) are stored in Amazon RDS for MySQL and retrieved by academic.php using the PHP Data Objects (PDO) extension. Skills, certifications and projects, which do not follow a fixed schema, are stored as a single item in an Amazon DynamoDB table and retrieved by skills.php using the AWS SDK for PHP. DynamoDB is accessed through the IAM role attached to the EC2 instance, so no AWS access keys are stored in the application. MySQL credentials are kept in a configuration file outside the public web root. The application is available at http://52.55.224.145/.

Note: The academic records come from the student's university grade sheet (27 courses, 92 credits, CGPA 9.13).

2. Objective

The objective is to build and deploy a cloud-hosted dynamic web application that integrates more than one AWS database service:

- Host a dynamic student portfolio on an Amazon EC2 instance.
- Provision and integrate an Amazon RDS for MySQL database.
- Retrieve academic records dynamically and display them on academic.php.
- Provision and integrate an Amazon DynamoDB table.
- Retrieve skills and certifications dynamically and display them on skills.php.
- Demonstrate working cloud database connectivity from code running on EC2.
- Provide a working public website that can be verified in a browser.

3. Technologies Used

Technology / Service	Role in the Project
Amazon EC2	Virtual server (t3.micro) that hosts the web application and runs the PHP code.
Ubuntu Linux 24.04	Operating system of the EC2 instance.
Apache HTTP Server	Serves the portfolio pages over HTTP on port 80.
PHP 8.3	Server-side language that generates the dynamic pages.
Amazon RDS for MySQL	Managed relational database storing the academic records.
Amazon DynamoDB	Managed NoSQL database storing skills, certifications and projects.
AWS SDK for PHP 3.400	Client library used by skills.php to call the DynamoDB API.
AWS IAM role (LabRole)	Role attached to the instance that grants DynamoDB access without keys.
Composer	PHP dependency manager used to install the AWS SDK.

Technology / Service	Role in the Project
PDO	Database abstraction layer for the MySQL connection and queries.
HTML / CSS / JavaScript	Interface, animations, filters and the arcade mini-game.
Amazon S3	Static website hosting for the separate S3 portfolio (Section 15).

4. System Architecture

The application follows a three-tier pattern. The presentation and application tiers run together on the EC2 instance, while the data tier is split across two managed AWS database services.

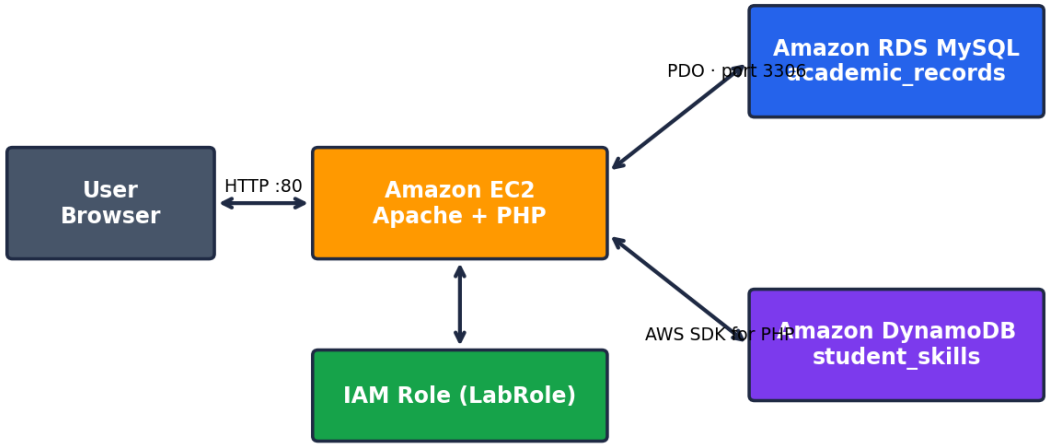


Figure 1: System architecture of the dynamic student portfolio.

- EC2 hosts the application; Apache serves the pages and passes PHP requests to the interpreter.
- academic.php connects to Amazon RDS MySQL through PDO and renders the rows as a quest log.
- skills.php connects to Amazon DynamoDB using the AWS SDK for PHP and renders skills, certifications and projects.
- An IAM role is used for AWS access instead of hard-coded credentials.

5. Amazon EC2 Configuration

Parameter	Value
Instance Name	student-portfolio-EC2
Instance ID	i-0752aeffdb13ef34b
Instance Type	t3.micro
Operating System	Ubuntu Server 24.04 LTS
Availability Zone	us-east-1b
Web Server / Runtime	Apache 2 with PHP 8.3 (libapache2-mod-php)
Public IPv4 Address	52.55.224.145
IAM Role	LabRole

Parameter	Value
Security Groups	launch-wizard-3 (web and SSH access); ec2-rds-2 (RDS link)
Website URL	http://52.55.224.145/

5.1 Deployment Process

- Launch the EC2 instance from an Ubuntu AMI with the IAM role attached.
- Configure the security group: SSH restricted to the administrator's IP, web access for visitors.
- Connect over SSH with the key pair created at launch.
- Install Apache, PHP, the MySQL extension, Composer and the MySQL client.
- Create the RDS database and the DynamoDB table.
- Copy the PHP files to /var/www/html, store the database configuration in /var/www/private, and install the AWS SDK with Composer.
- Load the SQL schema into RDS and seed the DynamoDB item.
- Test all three pages in a browser.

6. Portfolio Website

The site uses a retro game-inspired theme with a starfield background, a typing effect, animated counters and a built-in arcade mini-game. The navigation bar links to the Home page, the Quest Log (academic.php, backed by Amazon RDS) and the Skill Tree (skills.php, backed by Amazon DynamoDB).

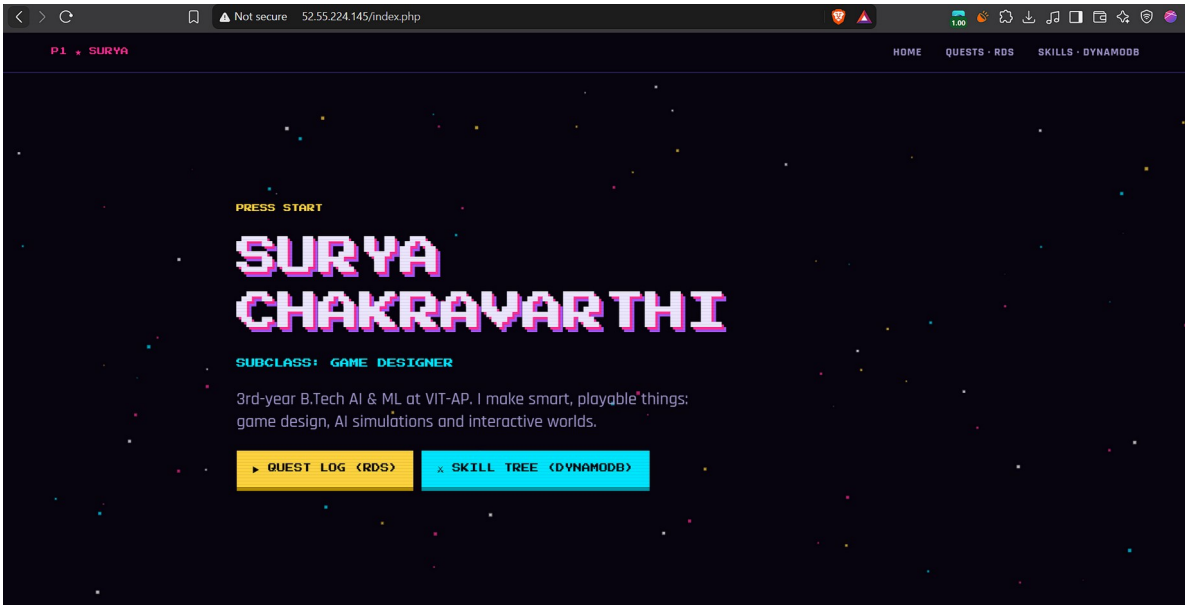


Figure 2: Portfolio home page served from Amazon EC2 (http://52.55.224.145/index.php).

7. Amazon RDS MySQL Configuration

Academic records follow a fixed schema with the same columns for every row, so a relational table is a natural fit. The database portfolio_db contains one table, academic_records.

Parameter	Value
DB Identifier	portfolio-db

Parameter	Value
Engine	MySQL Community
Instance Class	db.t3.micro
Availability Zone	us-east-1b
Public Access	No (reachable only from EC2)
Database / Table	portfolio_db / academic_records

7.1 SQL Table Structure

```
CREATE DATABASE IF NOT EXISTS portfolio_db;
USE portfolio_db;

CREATE TABLE academic_records (
  id       INT AUTO_INCREMENT PRIMARY KEY,
  code     VARCHAR(10)  NOT NULL,
  subject  VARCHAR(120) NOT NULL,
  credits  INT          NOT NULL,
  grade    VARCHAR(2)   NOT NULL,
  semester TINYINT      NOT NULL
);

INSERT INTO academic_records (code,subject,credits,grade,semester) VALUES
('CHY1009','Chemistry and Environmental Studies',4,'B',1),
('CSE1012','Problem Solving using Python',4,'A',1),
... 27 rows in total ...;
```

The RDS security group was created through the “connect to an EC2 compute resource” option, so inbound MySQL traffic on port 3306 is accepted from the EC2 instance only. The database is not exposed to the public internet.

8. RDS Connection in academic.php

The connection logic is separate from the presentation code, and the credentials are loaded from a file stored outside the document root.

```
$c = require '/var/www/private/db_config.php';

$pdo = new PDO(
  "mysql:host={$c['host']};dbname={$c['db']};charset=utf8mb4",
  $c['user'], $c['pass'],
  [ PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_TIMEOUT => 5 ]
);

$rows = $pdo->query(
  "SELECT code,subject,credits,grade,semester
   FROM academic_records ORDER BY semester,id"
)->fetchAll();
```

- Credentials live in /var/www/private/, which Apache does not serve.

- PDO uses exception-based error handling; a failed connection shows a red banner instead of a crash.
- Output is escaped with htmlspecialchars() before it is displayed.

The database host, user name and password are deliberately not shown. No passwords, AWS keys or private keys appear in this report.

9. Academic Records Webpage (Quest Log)

academic.php connects to Amazon RDS, runs the query shown in Section 8 and displays every course with its code, credits and grade. A green banner at the top confirms the connection. Grades are shown as rarity badges (S, A, B) and the list can be filtered by semester or searched by name.

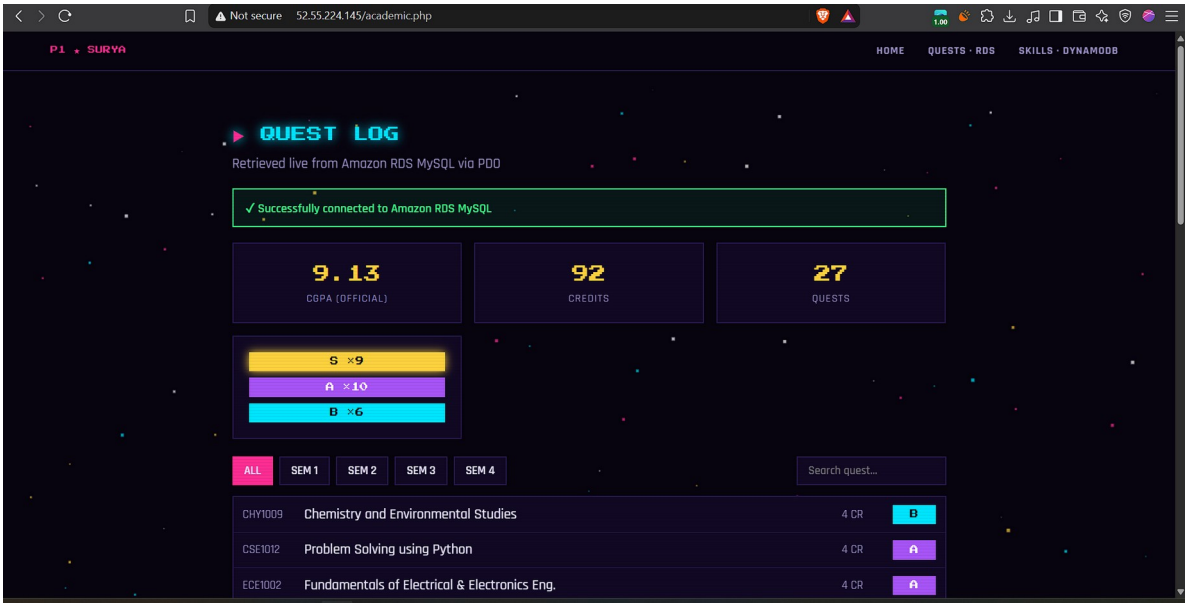


Figure 3: Academic records retrieved dynamically from Amazon RDS MySQL.

Summary item	Value	Source
Courses (quests)	27	academic_records table
Credits earned	92	Sum of credits column
S / A / B grades	9 / 10 / 6	Count by grade (2 courses are Pass-grade)
CGPA	9.13	University grade sheet (official)

10. Amazon DynamoDB Configuration

Skills, certifications and projects are variable-length lists with nested attributes rather than a fixed set of columns, so a schemaless key-value store suits them better than a relational table.

Parameter	Value
Table Name	student_skills
Partition Key	student_id (String)
Capacity Mode	On-demand
Region	us-east-1 (N. Virginia)

Parameter	Value
Items	1 (student_id = surya-chakravarthi)
Attributes	skills (list of maps), interests, certifications (list of maps), projects (list of maps)

The item is read using its partition key, so the lookup is a single-key query rather than a full table scan.

11. IAM Configuration

The EC2 instance uses the IAM role LabRole. This lets the AWS SDK running in the PHP application obtain temporary credentials from the instance metadata service at run time. No AWS access key or secret key is written into the PHP code or into any file on the web server.

12. AWS SDK for PHP

The AWS SDK for PHP (aws/aws-sdk-php, version 3.400.0) was installed with Composer. The DynamoDbClient handles request signing and endpoint resolution, so the client only needs the region.

```
require '/var/www/html/vendor/autoload.php';
use Aws\DynamoDb\{DynamoDbClient,Marshaler};

$db = new DynamoDbClient([
  'version' => 'latest',
  'region'  => 'us-east-1'
]);

$r = $db->query([
  'TableName' => 'student_skills',
  'KeyConditionExpression' => 'student_id = :s',
  'ExpressionAttributeValues' => [
    ':s' => ['S' => 'surya-chakravarthi']
  ]
]);

$item = (new Marshaler())->unmarshalItem($r['Items'][0]);
```

The code reads the skills, certifications and projects attributes and renders them dynamically, so a skill added in DynamoDB appears on the website without any code change.

13. Skills Webpage (Skill Tree)

skills.php retrieves the stored item from Amazon DynamoDB and displays the skills as grouped level bars, followed by the certification inventory and the project list. A green banner confirms that the connection succeeded, and a filter box narrows the skills as the visitor types.

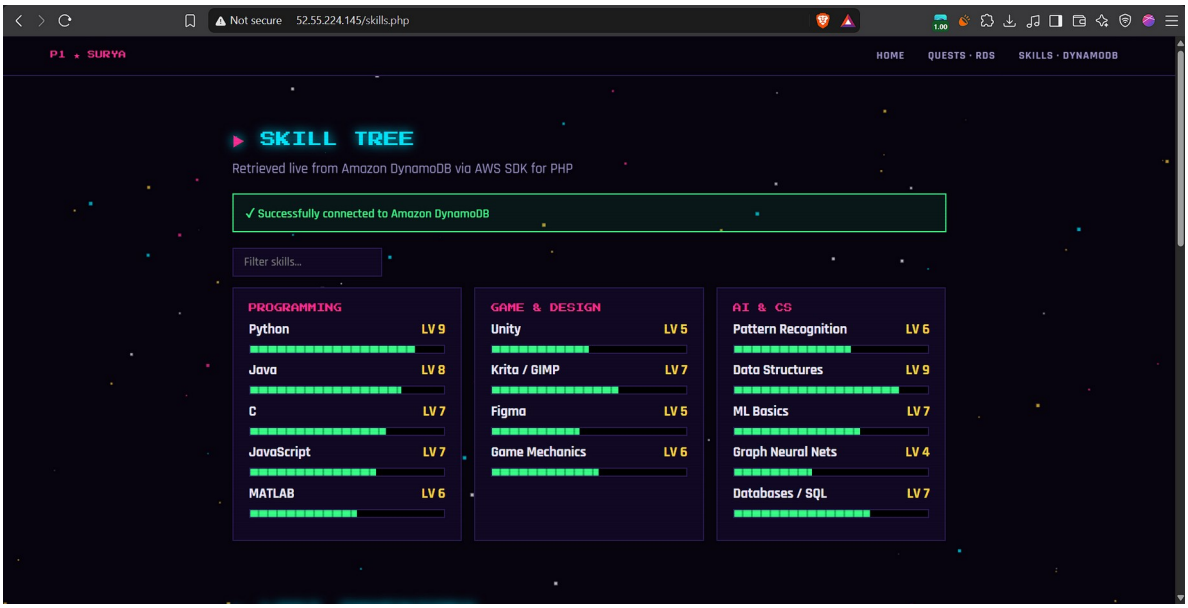


Figure 4: Skills dynamically retrieved from Amazon DynamoDB.

14. AWS Console Evidence

14.1 Amazon EC2

The instance student-portfolio-EC2 is Running, is of type t3.micro, has passed all 3/3 status checks, has the IAM role LabRole attached and uses two security groups.

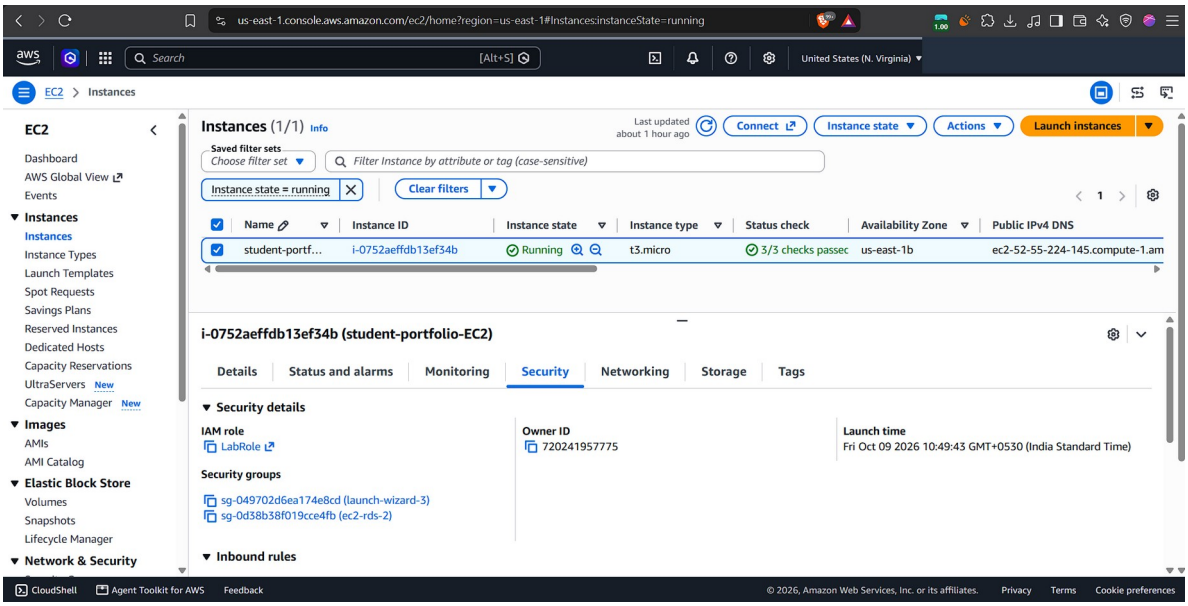


Figure 5: EC2 instance details with the IAM role and security groups.

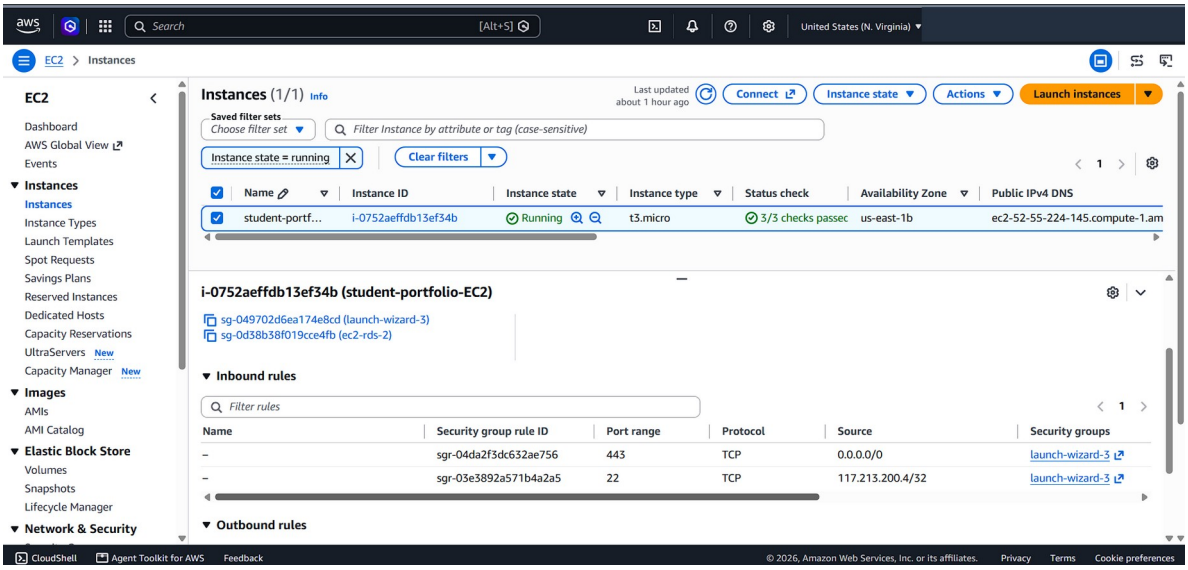


Figure 6: Inbound rules of the EC2 security group (SSH limited to the administrator's IP).

14.2 Amazon RDS

The portfolio-db instance is Available on MySQL Community, class db.t3.micro. (The second entry, db, was being deleted and is not used.)

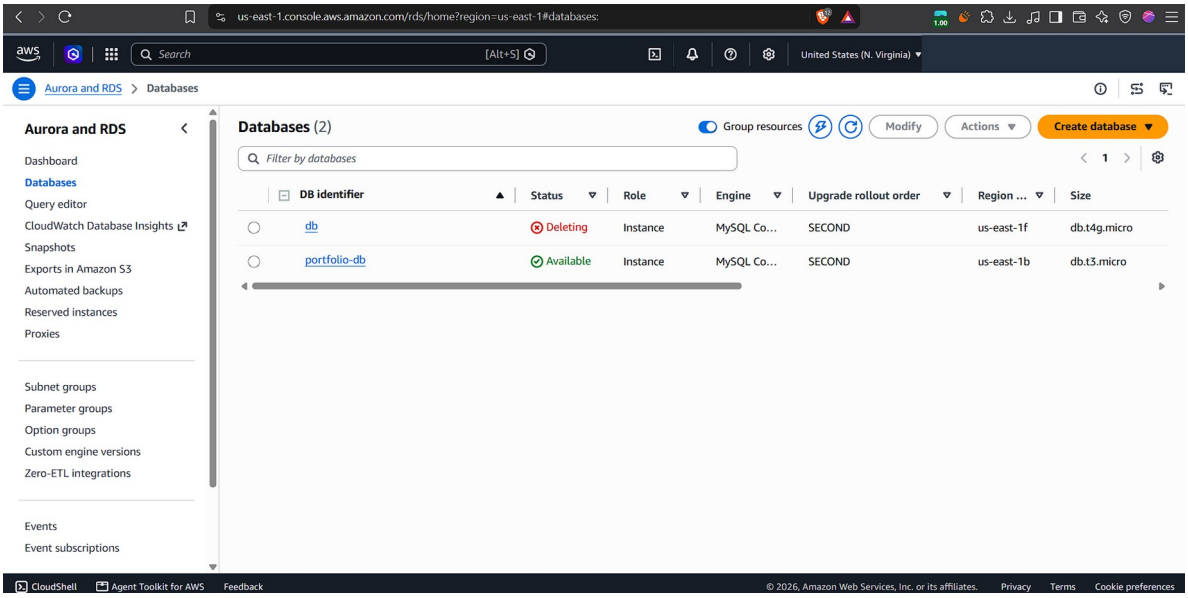


Figure 7: Amazon RDS database portfolio-db in the Available state.

14.3 Amazon DynamoDB

The student_skills table is Active with the partition key student_id (String) and on-demand capacity.

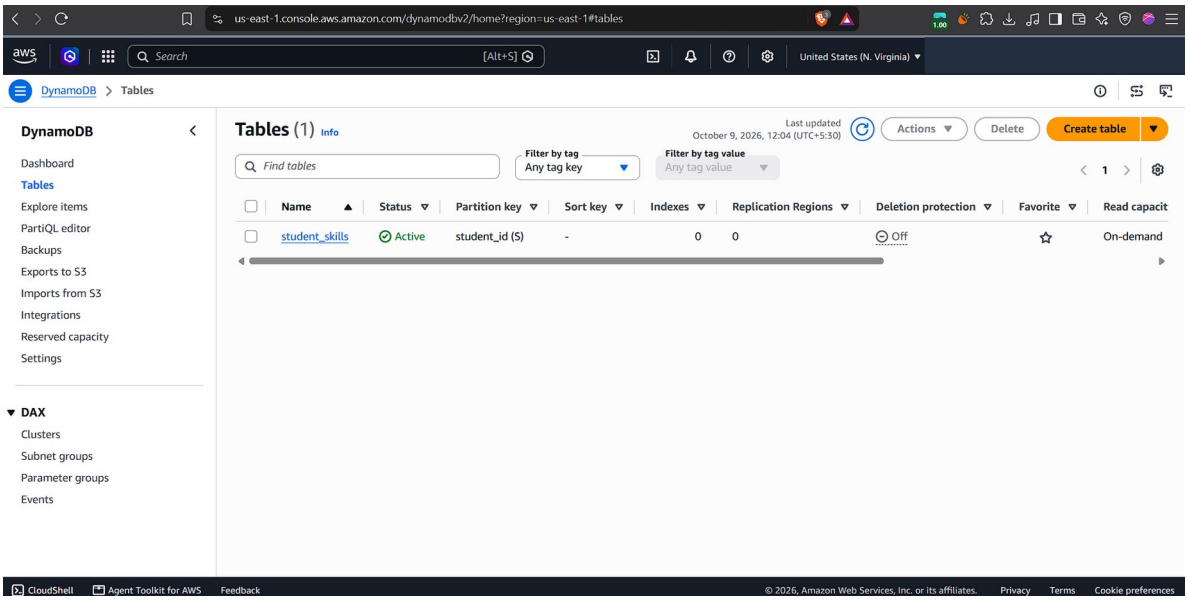


Figure 8: DynamoDB table student_skills.

15. Personal Portfolio on Amazon S3 Static Website Hosting

A separate personal portfolio was also deployed as a static website on Amazon S3. The bucket surya-gaming-portfolio-2026 has static website hosting enabled with index.html as the index document, and a bucket policy allows public read access to the objects. Unlike the EC2 application, this site is purely static and uses no database.

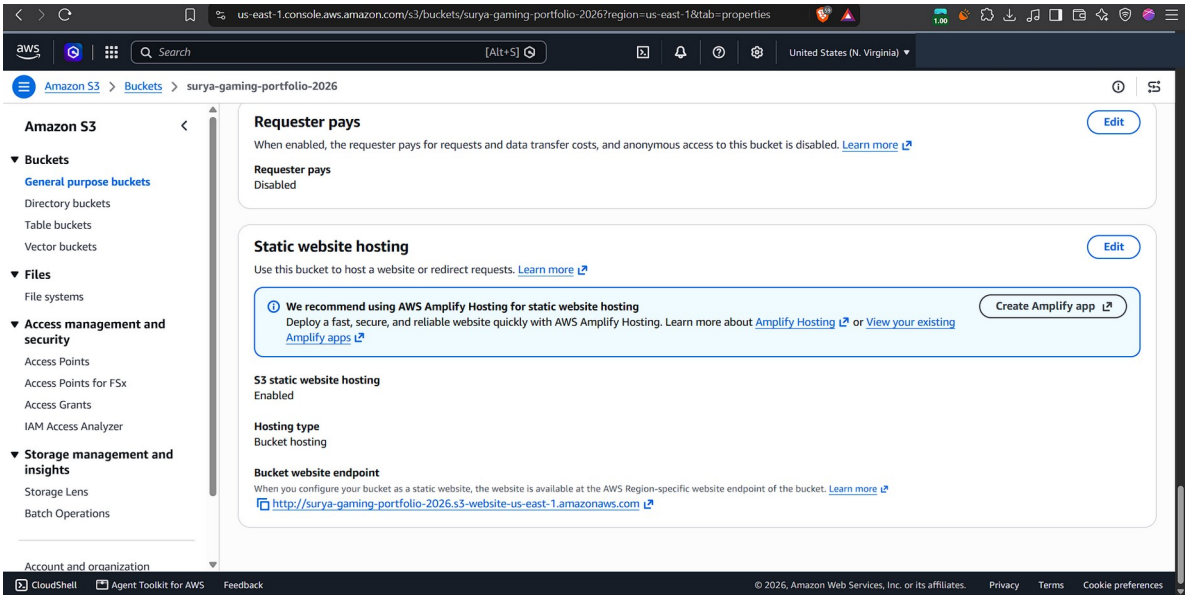


Figure 9: S3 static website hosting enabled on bucket surya-gaming-portfolio-2026.

16. Database Connectivity Flow

For academic.php:

```
Browser -> EC2 Apache/PHP -> PDO -> Amazon RDS MySQL
        -> academic_records -> PHP webpage
```

For skills.php:

```
Browser -> EC2 Apache/PHP -> AWS SDK for PHP -> Amazon DynamoDB
        -> student_skills -> PHP webpage
```

17. Testing and Results

The deployed application was verified with a web browser and the AWS Management Console.

Test Case	Expected Result	Actual Result	Status	Evidence
EC2 website access	Portfolio loads publicly	Loaded	PASS	Fig. 2
RDS connection	academic.php connects to MySQL	Green banner shown	PASS	Fig. 3
Academic data retrieval	27 records displayed	Displayed	PASS	Fig. 3
DynamoDB connection	skills.php accesses DynamoDB	Green banner shown	PASS	Fig. 4
Skills retrieval	Skills, certificates, projects shown	Displayed	PASS	Fig. 4
Public URL	Reachable via EC2 public IP	Accessible	PASS	Fig. 2
S3 static hosting	Website endpoint enabled	Enabled	PASS	Fig. 9

All test cases passed, which confirms that both database integrations work from the public website.

18. Security Considerations

- Amazon RDS accepts MySQL connections only from the EC2 security group and has public access disabled.
- AWS access keys are not hard-coded anywhere in the PHP source.
- DynamoDB access uses the IAM role LabRole, which supplies short-lived credentials.
- The database configuration file is stored outside the web root and is not served by Apache.
- SSH access is restricted to the administrator’s own IP address.
- All database output is escaped with htmlspecialchars() to prevent cross-site scripting.
- Secrets are not committed to source code and are not reproduced in this report.

19. Result

The dynamic student portfolio was deployed on Amazon EC2 and integrated with both Amazon RDS MySQL and Amazon DynamoDB. Academic records are retrieved from the relational database on academic.php, and skills, certifications and projects are retrieved from the NoSQL database on skills.php. Both pages report a successful connection and all tests passed. The live website is available at http://52.55.224.145/.

20. Conclusion

This project demonstrates a cloud-hosted dynamic web application that uses multiple AWS database services, each matched to its data. Amazon RDS MySQL holds the structured academic records, while Amazon DynamoDB holds the flexible skills, certifications and projects data retrieved by partition key. Amazon EC2 hosts the Apache and PHP tier that joins the two data sources, and IAM with the AWS SDK for PHP provides secure integration without long-lived credentials in the code. The assignment covers both the practical steps of deploying a web application on AWS and the reasoning for choosing a relational or a NoSQL database.

21. Future Enhancements

- Add user authentication and an admin dashboard for managing portfolio content.
- Implement full create, update and delete operations for both the RDS and DynamoDB data.
- Register a domain name and enable HTTPS with a TLS certificate (for example through CloudFront).
- Introduce automated deployment so code changes are released without manual copying.
- Add the Financial Fraud Detection using GNN project details once the work is complete.
- Improve responsive behaviour for small mobile screens.